INTERNSHIP REPORT

User Role Management Portal

Days 1 – 12 | May 18 – 29, 2026

Intern: Rushil Parikh

ASP.NET MVC | C# | PostgreSQL | Bootstrap 5 | Entity Framework

CONFIDENTIAL

WEEK 1

Date	Day Theme	Activities	Key Takeaway
18 May (Mon)	**Departmental Introductions**	Introductory sessions with Heads of Department: PKI / DSC, QA & Testing, HR, Mining Solutions, Finance & Accounts. Briefed on (n)Code Solutions' product portfolio and organizational structure & sevices.	Understood how each department contributes to (n)Code's end-to-end digital solutions, from certificate issuance (PKI) to procurement platforms.
19 May (Tue)	**Product Exposure & Data Centre Tour**	Hands-on walkthrough of two flagship web platforms: • eAuction — online bidding and auction management • nProcure — end-to-end e-procurement portal Visit to GNFC INFO Tower: toured the Data Centre including electrical room, control & data center floor, and telecommunication room. Observed full redundancy setup — every system has backup.	Learned how enterprise-grade data centers implement redundancy at every layer (power, network, cooling) to ensure zero-downtime SLAs.
20 May (Wed)	**Project Allocation Meeting**	Attended a formal project allocation meeting. Presented with the internship project: Ticket Generation and Management System. Discussed scope, modules, and expected deliverables with the project mentor.	Understood the full scope of the Ticket Management project: user registration, login, role-based access, ticket CRUD, and status workflow.
21 May (Thu)	**Industrial Coding Standards**	Guided walkthrough of the company's existing production codebase. Shown coding conventions: naming, layered architecture (Controller → Service → Repository), commenting standards, and Git workflow. Hands-on task: write a sample module following their standards in Visual Studio Community.	Experienced the difference between academic code and production code — importance of consistent naming, separation of concerns, and code review readiness.

WEEK 1 DAY 5

1. Intern Information

Intern Name	Rushil Parikh
Date	May 22, 2026
Day	Week 1, Day 5 of Internship
Project Title	User Role Management Portal
Tech Stack	ASP.NET MVC | C# | PostgreSQL | Bootstrap | Entity Framework
Reporting To	Hemal Shah
Work Mode	VS Community, PostgreSQL

2. Day Objective — Day 1

The objective of Day 1 was to understand the overall project requirements, set up the development environment, and begin learning foundational database concepts. This included understanding the User Role Management Portal architecture, learning PostgreSQL basics, and studying how databases are structured for user and role management.

3. Work Completed Today — Day 1

3.1 Project Requirements Understanding

- Reviewed detailed process flow: User Registration → Login → Dashboard → User Management (Admin Only)
- Understood key components: Registration Page with form validation, Login Page with credential verification, Dashboard for authenticated users, User List page with admin-only access
- Identified database entities: RoleMaster table (Admin, User roles), UserMaster table (user credentials, roles, timestamps)
- Mapped requirements to technical implementation: form validation, password hashing, role-based authorization

3.2 PostgreSQL Installation and Setup

- Downloaded PostgreSQL 15 Community Edition from official PostgreSQL website
- Followed step-by-step installation process with default settings (Port: 5432, Username: postgres)
- Set secure password for PostgreSQL superuser account
- Verified installation by opening pgAdmin 4 and confirming server connectivity
- Explored pgAdmin interface: Server management, database browser, query tool functionality

3.3 Project Database Design Planning

- Designed RoleMaster table structure: RoleId (Primary Key), RoleName (Admin/User), CreatedDate, UpdatedDate
- Designed UserMaster table structure: UserId (Primary Key), FullName, Username, Email, Password, RoleId (Foreign Key), CreatedDate, UpdatedDate, IsActive
- Documented foreign key relationship: UserMaster.RoleId → RoleMaster.RoleId for maintaining referential integrity
- Planned indexing strategy: Primary keys on Id columns, indexes on Username and Email for login queries

4. Key Learnings — Day 1

- Relational Database Concepts: Understanding how tables relate to each other through primary and foreign keys, and how normalization eliminates data redundancy while maintaining consistency.
- Database Schema Design: Proper table design with appropriate data types, constraints, and relationships provides the foundation for building scalable, maintainable applications.
- Primary and Foreign Keys: Primary keys uniquely identify records, while foreign keys create references between tables, enabling relational integrity and preventing orphaned records.

5. Task Status Summary — Day 1

#	Task	Category	Status
1	ASP.NET Core MVC project requirements understood	Analysis	**Completed**
2	PostgreSQL 15 installed and configured	Setup	**Completed**
3	pgAdmin connectivity verified	Setup	**Completed**
4	RoleMaster table structure planned	Database	**Completed**
5	UserMaster table structure planned	Database	**Completed**
6	Foreign key relationship UserMaster → RoleMaster designed	Database	**Completed**
7	Database indexing strategy documented	Database	**Completed**

WEEK 2 DAY 1

1. Intern Information

Intern Name	Rushil Parikh
Date	May 23, 2026
Day	Week 2, Day 1 of Internship
Project Title	User Role Management Portal
Tech Stack	ASP.NET MVC | C# | PostgreSQL | Bootstrap | Entity Framework
Reporting To	Hemal Shah
Work Mode	VS Community, PostgreSQL

2. Day Objective — Day 2

The objective of Day 2 was to continue learning foundational concepts by studying C# programming language basics and creating the PostgreSQL database with tables for the User Role Management Portal. This included learning C# syntax, data types, control structures, and implementing database tables in PostgreSQL with proper structure and relationships.

3. Work Completed Today — Day 2

3.1 C# Programming Fundamentals

- Learned C# basic syntax: variables, data types (string, int, bool, decimal), type declarations, and type inference
- Studied control structures: if-else statements, switch cases, loops (for, while, foreach) for conditional execution
- Understood object-oriented programming concepts: classes, properties, methods, constructors, and encapsulation
- Learned accessibility modifiers: public, private, protected for controlling member access and encapsulation
- Practiced writing simple C# console programs: variable assignments, arithmetic operations, string manipulation, and loops.

3.2 PostgreSQL Database Creation

- Created UserPortalDB database using pgAdmin Query Tool with proper collation and encoding (UTF-8)
- Created RoleMaster with columns: RoleId (SERIAL PRIMARY KEY), RoleName (VARCHAR 100), CreatedDate (TIMESTAMP DEFAULT NOW())
- Inserted initial role data: 'Admin' and 'User' roles with automatic timestamps from database server
- Verified table creation and data insertion using SELECT queries in pgAdmin Query Tool

3.3 Visual Studio Community Setup

- Downloaded Visual Studio Community 2022 (free edition) with .NET development workloads
- Installed with workloads: ASP.NET and web development, .NET desktop development, data storage and processing
- Launched Visual Studio and configured initial settings for C# development
- Familiarized with IDE interface: Solution Explorer for project structure, Code Editor for writing code, Properties window for object details, Output window for diagnostics
- Completed project template exploration: understanding available project types (Console, Web, Desktop applications)

4. Key Learnings — Day 2

- C# Type System: C# is a strongly-typed language where every variable must have a declared type, providing compile-time safety and enabling the compiler to catch type-related errors before runtime.
- Object-Oriented Programming: Classes in C# encapsulate data (properties) and behavior (methods), providing a blueprint for creating objects that model real-world entities and their interactions.
- Database Constraints: PRIMARY KEY, FOREIGN KEY, NOT NULL, and DEFAULT constraints enforce data quality and consistency at the database level, preventing invalid or incomplete data from being stored.
- Timestamp Management: Using database-level timestamp defaults (TIMESTAMP DEFAULT NOW()) ensures consistent, server-based time tracking independent of client system clocks, preventing synchronization issues.
- IDE Proficiency: Visual Studio's rich interface provides tools for code editing, debugging, project management, and database connectivity, significantly improving developer productivity and code quality.

5. Task Status Summary — Day 2

#	Task	Category	Status
1	C# programming fundamentals learned	Learning	**Completed**
2	Object-oriented programming concepts practiced	Learning	**Completed**
3	UserPortalDB database created in PostgreSQL	Database	**Completed**
4	RoleMaster table created	Database	**Completed**
5	Admin and User roles inserted	Database	**Completed**
6	Visual Studio Community 2022 installed	Setup	**Completed**
7	ASP.NET development workloads configured	Setup	**Completed**

WEEK 2 DAY 2

1. Intern Information

Intern Name	Rushil Parikh
Date	May 26, 2026
Day	Week 2, Day 2 of Internship
Project Title	User Role Management Portal
Tech Stack	ASP.NET MVC | C# | PostgreSQL | Bootstrap | Entity Framework
Reporting To	Hemal Shah
Work Mode	VS Community, PostgreSQL

2. Day Objective — Day 3

The objective of Day 3 was to understand the ASP.NET MVC architecture pattern and Entity Framework concepts. This included learning how the Model-View-Controller pattern separates concerns, understanding how Entity Framework maps C# objects to database tables, learning about async/await patterns, and creating an ASP.NET MVC project with database connectivity for the User Role Management Portal.

3. Work Completed Today — Day 3

3.1 ASP.NET MVC Architecture Learning

- Studied Model-View-Controller design pattern: separation of concerns between data, presentation, and business logic
- Understood Model layer: C# classes representing database entities (User, Role) with properties and validation attributes
- Learned Controller layer: handling HTTP requests, processing business logic, querying data, determining responses
- Studied View layer: Razor templating syntax for generating HTML, form binding with models, conditional rendering based on data
- Reviewed complete request flow: Browser HTTP Request → Route Matching → Controller Action Invocation → View Rendering → HTML Response

3.2 Entity Framework Core Concepts

- Understood ORM (Object-Relational Mapping): automatically mapping C# classes to database tables without writing raw SQL
- Learned DbContext: central class that manages database connections, entity state tracking, and query translation to SQL
- Studied DbSet<T>: generic collections representing database tables and providing LINQ query capabilities
- Understood Entity Fluent API: configuring table names, column types, relationships, constraints, and behaviors in code
- Learned migrations: code-first database schema management with version control and reversible changes
- Practiced async/await patterns: writing non-blocking database queries with Task<T> return types for better scalability

3.3 .NET Framework and ASP.NET MVC Project Structure

- Understood ASP.NET MVC project folder organization: Controllers/ for request handlers, Views/ for Razor templates, Models/ for entities, wwwroot/ for static files
- Studied Startup.cs/Program.cs: dependency injection configuration, middleware pipeline setup, service registration
- Learned routing configuration: how URLs are mapped to controller actions via conventions and attributes (RouteValues)
- Understood NuGet package management: adding external dependencies like EntityFrameworkCore, PostgreSQL provider, Bootstrap, jQuery

3.4 ASP.NET MVC Project Creation and Configuration

- Created new ASP.NET Core MVC project in Visual Studio: UserPortal project with appropriate template
- Selected project template with authentication scaffolding support and modern C# features
- Installed required NuGet packages: EntityFrameworkCore, EntityFrameworkCore.PostgreSQL, Bootstrap 4, jQuery
- Configured appsettings.json: PostgreSQL connection string with server address, database name, username, password
- Set up ApponDbContext class inheriting from Microsoft.EntityFrameworkCore.DbContext for entity management

3.5 Registration and Login Flow Understanding

- Studied complete registration flow: User Form Submission → Input Validation → Password Hashing → Database Insert → Success/Error Response
- Understood password security: never storing plain text passwords, using bcrypt hashing with salt to prevent brute-force attacks
- Learned login flow: Credential Submission → Username Database Lookup → Hash Comparison → Session Creation → Dashboard Redirect
- Studied session management: storing user information in session variables, session expiration policies, security considerations
- Understood role-based authorization: checking user role before allowing access to protected resources (User List = Admin Only)

4. Key Learnings — Day 3

- MVC Architectural Pattern: Separating Model (data), View (presentation), and Controller (logic) makes applications maintainable, testable, and allows different team members to work on each layer independently.
- Entity Framework as Abstraction: ORM frameworks like EF eliminate the need to write manual SQL CRUD operations, reducing boilerplate code and SQL injection vulnerabilities while maintaining type safety.
- DbContext as Unit of Work: DbContext tracks entity changes in memory and batches them into a single database transaction, ensuring data consistency and improving performance by reducing round-trips to the database.
- Migrations for Schema Management: Database migrations provide version control for schema changes, enabling team collaboration and easy rollback to previous schema versions if needed.
- Async/Await for Scalability: Asynchronous database operations free up threads while waiting for I/O, allowing a single server to handle more concurrent requests than synchronous blocking operations.
- Authentication vs Authorization: Authentication verifies who the user is (login), while authorization determines what they can access (roles), and both are essential for secure applications.

5. Task Status Summary — Day 3

#	Task	Category	Status
1	ASP.NET MVC architecture studied	Backend	**Completed**
2	Entity Framework Core concepts learned	Backend	**Completed**
3	DbContext and DbSet configuration understood	Backend	**Completed**
4	ASP.NET Core MVC project created	Setup	**Completed**
5	PostgreSQL connection string configured	Setup	**Completed**
6	ApponDbContext created	Backend	**Completed**
7	Registration and login flow architecture understood	Authentication	**Completed**

WEEK 2 DAY 3

1. Intern Information

Intern Name	Rushil Parikh
Date	May 27, 2026
Day	Week 2, Day 3 of Internship
Project Title	User Role Management Portal
Tech Stack	ASP.NET MVC | C# | PostgreSQL | Bootstrap | Entity Framework
Reporting To	Hemal Shah
Work Mode	VS Community, PostgreSQL

2. Day Objective — Day 4

The objective of Day 4 was to begin hands-on implementation by creating models, configuring the database context, applying migrations to create actual database tables, and building the registration functionality including controllers, views, form validation, and Bootstrap styling. This day transitioned from learning concepts to writing actual working code.

3. Work Completed Today — Day 4

3.1 User Model Creation with Data Validation

- Created User model class with properties: UserId, FullName, Username, Email, Password, RoleId, CreatedDate, UpdatedDate, IsActive
- Implemented data validation attributes: [Required] for mandatory fields, [EmailAddress] for valid email format, [MinLength] and [MaxLength] for string constraints
- Added navigation property: public Role Role { get; set; } for relational mapping to RoleMaster
- Used [NotMapped] attribute: ConfirmPassword property for registration form (accepted but not stored in database)
- Configured column attributes: [Key] for primary key, [ForeignKey] for relationship specification, [Column] for name mapping

3.2 Role Model and DbContext Configuration

- Created Role model class representing RoleMaster: RoleId (int), RoleName (string), CreatedDate, UpdatedDate
- Configured DbContext with DbSet<User> and DbSet<Role> collections for LINQ querying
- Set up foreign key relationship in OnModelCreating: explicitly mapping UserMaster.RoleId → RoleMaster.RoleId with DeleteBehavior.Cascade
- Added table name mapping: configuring "RoleMaster" and "UserMaster" table names to match existing database schema
- Configured required string lengths and other constraints via Fluent API in OnModelCreating override.

3.3 Database Migrations and Connection Setup

- Added PostgreSQL provider to services in Startup: services.AddDbContext<ApponDbContext>(options => options.UseNpgsql(connectionString))
- Created initial migration using Package Manager Console: Add-Migration InitialCreate command
- Reviewed generated migration file: verified CreateTable statements, constraints, foreign keys match intended schema
- Applied migration to PostgreSQL database: Update-Database command to execute migration scripts
- Verified tables and schema in pgAdmin: confirmed columns, data types, constraints created correctly in PostgreSQL.

3.4 AccountController Registration Action (GET)

- Created AccountController class inheriting from Controller for handling user account operations
- Implemented Register GET action: returns Register view with empty User model for form display
- Applied [HttpGet] attribute: explicitly mapping GET requests to this action method
- Return statement: return View(); returns Register.cshtml view to display form to user browser

3.5 AccountController Registration Action (POST)

- Implemented Register POST action: [HttpPost] attribute routes form submissions to this method
- Added [ValidateAntiForgeryToken]: prevents CSRF attacks by validating token submitted with form
- Model validation check: if (!ModelState.IsValid) return View(model); redisplays form with validation error messages
- Duplicate username prevention: querying database to check if username already exists before registration
- Password hashing implementation: using BCrypt.Net.BCrypt.HashPassword(user.Password) to securely hash password with salt
- Database insertion: _context.Users.Add(user) adds user to context, _context.SaveChangesAsync() persists to database
- Redirect on success: return RedirectToAction("Login"); redirects user to login page after successful registration

3.6 Register View Creation with Razor and Bootstrap

- Created Register.cshtml view: strongly typed to User model with @model User directive at top
- Built HTML form with Bootstrap styling: form-group class for grouping, form-control class for consistent input styling
- Used ASP.NET Tag Helpers: asp-for="Model.Property" for two-way model binding, asp-action and asp-controller for form routing
- Added form fields: FullName, Username, Email, Password, ConfirmPassword with appropriate input types
- Implemented client-side validation: HTML5 required, email, minlength attributes for immediate user feedback
- Added validation error display: asp-validation-summary="All" shows server-side validation errors, asp-validation-for shows field-specific errors

4. Key Learnings — Day 4

- Data Annotations for Validation: Attributes like [Required], [EmailAddress], [MinLength] provide declarative validation rules that work automatically on both client side and server side without additional code.
- Navigation Properties for Relationships: In EF, navigation properties enable accessing related entities without writing JOIN queries, providing a natural object-oriented way to traverse relationships.
- Model Binding in ASP.NET: When a form is submitted, ASP.NET automatically binds form data to model properties by name, simplifying server-side code and reducing manual property assignment.
- Tag Helpers for Strongly-Typed Views: ASP.NET Tag Helpers like asp-for, asp-action, asp-validation-for provide IntelliSense support and compile-time checking, preventing runtime errors from typos in property names.
- Bootstrap CSS Framework: Bootstrap provides pre-built responsive components and utility classes that enable creating professional-looking, mobile-friendly UIs without writing custom CSS.

5. Task Status Summary — Day 4

#	Task	Category	Status
1	UserMaster model created with validations	Database	**Completed**
2	RoleMaster model created	Database	**Completed**
3	ApponDbContext configured with DbSets	Backend	**Completed**
4	Foreign key relationship configured	Database	**Completed**
5	EF Core migration created and applied	Database	**Completed**
6	AccountController registration actions implemented	Backend	**Completed**
7	Register.cshtml view created using Bootstrap	Frontend	**Completed**

WEEK 2 DAY 4

1. Intern Information

Intern Name	Rushil Parikh
Date	May 28, 2026
Day	Week 2, Day 4 of Internship
Project Title	User Role Management Portal
Tech Stack	ASP.NET MVC | C# | PostgreSQL | Bootstrap | Entity Framework
Reporting To	Hemal Shah
Work Mode	VS Community, PostgreSQL

2. Day Objective — Day 5

The objective of Day 5 was to complete the core functionality of the User Role Management Portal by implementing the login system, dashboard, user list page with role-based access control, and comprehensive testing. This day focused on completing all user-facing features, verifying functionality end-to-end, and debugging any issues found during testing.

3. Work Completed Today — Day 5

3.1 Login Page Implementation

- Created Login.cshtml view with clean, user-friendly form layout
- Built form fields: Username (text input) and Password (password input) with proper labels
- Implemented Razor syntax with ASP.NET Tag Helpers for model binding and action routing
- Applied Bootstrap styling: centered layout with col-md-4 for responsive width, form-group spacing, consistent form-control styling
- Added client-side validation: HTML5 required attributes for mandatory fields
- Included helpful link: "Don't have an account? Register here" linking to registration page

3.2 AccountController Login Actions (GET and POST)

- Implemented Login GET action: [HttpGet] attribute returns login view for displaying form
- Created Login POST action: [HttpPost] [ValidateAntiForgeryToken] attributes for form submission handling

- Added model validation: if (!ModelState.IsValid) return View(model); redisplays form with error messages
- Database username lookup: _context.Users.FirstOrAsync(u => u.Username == username) searches for user by username
- Session creation: storing critical user information - HttpContext.Session.SetString("UserId", user.UserId.ToString()); for subsequent requests
- Success flow: credentials correct → session populated → return RedirectToAction("Dashboard", "Dashboard");
- Error handling: invalid credentials display error message without revealing which field is incorrect (security best practice)

3.3 Dashboard Controller and View

- Created DashboardController class with authorization requirement: [Authorize] attribute on class level
- Implemented Dashboard GET action: retrieves authenticated user information from session
- Fetches user from database: using session UserId to load complete user record with role details
- Created Dashboard.cshtml view: displays welcome message with user's full name retrieved from model
- Added user profile section: shows username, email, and assigned role information in formatted layout
- Included navigation links: link to User List (for admin users only) and Logout functionality
- Session validation: checking if UserId exists in session before displaying dashboard, redirecting to login if missing

3.4 User List Page Implementation (Admin Only)

- Created UserList view in DashboardController for displaying all registered users
- Implemented role-based access control: checking if user.RoleId equals AdminRoleId before displaying page
- Non-admin redirect: if user is not admin, redirecting to dashboard with TempData message: "Access Denied"
- Data query: _context.Users.Include(u => u.Role).ToListAsync() retrieves all users with role information
- Built data display table: columns for FullName, Username, Email, Role, and CreatedDate
- Bootstrap table styling: table class for basic styling, table-striped for alternating row colors, table-hover for interactivity
- Responsive design: table-responsive class wraps table for mobile devices

3.5 Testing Registration and Login Flow

- Started application: F5 in Visual Studio launched application on http://localhost:5044
- Tested registration page: filled form with valid data (name, username, email, password)
- Submitted registration form: successfully processed without errors
- Verified database insertion: checked UserMaster table in pgAdmin for new user record
- Confirmed password hashing: verified password is stored as bcrypt hash (e.g., $2a$11$...), not plain text
- Tested login with new user: entered correct username and password successfully logged in
- Tested login with invalid credentials: incorrect password/username displays error message
- Verified session creation: confirmed UserId, Username, RoleId stored in session after login
- Dashboard display: user redirected to dashboard showing personalized welcome message and profile info

3.6 Role-Based Access Control Testing

- Registered test admin user with Admin role manually assigned in database
- Verified admin access: admin user successfully accessed User List page
- Confirmed data display: User List page displays all registered users in table format
- Registered regular user with User role (non-admin)
- Tested access denial: non-admin user attempting to access User List redirected to dashboard
- Verified redirect message: TempData displayed "Access Denied" message on dashboard
- Tested menu visibility: User List link hidden from navigation for non-admin users
- Confirmed role display: roles correctly displayed next to users in User List for admin view

4. Key Learnings — Day 5

- Session Management in ASP.NET: Sessions store user-specific data on the server across multiple HTTP requests, eliminating the need to re-query the database on every request for user information.
- Role-Based Authorization: Using role information from the database to control access to features ensures authorization is flexible, updatable, and doesn't require code changes when role permissions change.
- Authentication vs Authorization: Authentication (login) verifies user identity, while authorization (role checks) determines what authenticated users can access, and both layers are necessary for security.
- End-to-End Testing: Testing the complete user journey (registration → login → dashboard → restricted access) reveals integration issues that unit tests might miss, such as session not persisting across redirects.
- Database Data Integrity: Proper foreign key constraints and NOT NULL requirements at the database level prevent invalid data states that could cause runtime errors in the application.
- Improved UI: Website UI including buttons, loading pages, animation added to make website portal easy to use and navigate.

5. Task Status Summary — Day 5

#	Task	Category	Status
1	Login page implemented with Bootstrap UI	Frontend	**Completed**
2	Authentication and session management completed	Authentication	**Completed**
3	DashboardController and Dashboard view created	Backend	**Completed**
4	Admin-only User List page implemented	Backend	**Completed**
5	Role-based access control implemented	Security	**Completed**
6	End-to-end registration and login testing completed	Testing	**Completed**
7	UI improvements and navigation enhancements added	Frontend	**Completed**